Data Structures & Algorithms

Week 1 Assignment – DSA Fundamentals & Arrays

Language: C++

No.	Assignment	Technique	Time Complexity
1	Linear Search	Check each element	O(n)
2	Binary Search	Halve a sorted range	O(log n)
3	Bubble Sort + swap count	Swap adjacent pairs	O(n^2)
4	Reverse array (no extra array)	Two pointers	O(n), space O(1)
Mini	Student Marks Analyzer	Sort + sum	O(n^2) with bubble sort

Assignment 1: Linear Search

Idea: Start at index 0 and compare each element with the key until it is found or the array ends. Works on any array (sorted or not). Best case O(1), worst case O(n).

Program

```
#include <iostream>
using namespace std;

// Linear Search: check every element one by one.
// Returns index of key, or -1 if not found.
int linearSearch(int arr[], int n, int key) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == key)
            return i;
    }
    return -1;
}

int main() {
    int n, key;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[100];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    cout << "Enter element to search: ";
    cin >> key;

    int idx = linearSearch(arr, n, key);
    if (idx != -1)
        cout << "\nElement " << key << " found at index " << idx
             << " (position " << idx + 1 << ")" << endl;
    else
        cout << "\nElement " << key << " not found in the array." << endl;
    return 0;
}
```

Example 1 – key present

```
Enter number of elements: 6
Enter 6 elements: 12 45 7 89 23 56
Enter element to search: 89

Element 89 found at index 3 (position 4)
```

Example 2 – key absent

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50
Enter element to search: 35

Element 35 not found in the array.
```

Assignment 2: Binary Search on a Sorted Array

Idea: Compare the key with the middle element. If equal, done; if the key is larger, discard the left half; otherwise discard the right half. Each step halves the range, so the time is O(log n). The array must be sorted.

Program

```
#include <iostream>
using namespace std;

// Binary Search: array MUST be sorted (ascending).
// Repeatedly halve the search range.
int binarySearch(int arr[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;   // avoids overflow
        if (arr[mid] == key)
            return mid;
        else if (arr[mid] < key)
            low = mid + 1;                  // search right half
        else
            high = mid - 1;                 // search left half
    }
    return -1;
}

int main() {
    int n, key;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[100];
    cout << "Enter " << n << " SORTED elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];
    cout << "Enter element to search: ";
    cin >> key;

    int idx = binarySearch(arr, n, key);
    if (idx != -1)
        cout << "\nElement " << key << " found at index " << idx << endl;
    else
        cout << "\nElement " << key << " not found." << endl;
    return 0;
}
```

Dry run: arr = {5, 12, 19, 27, 34, 48, 63}, key = 34

Step	low	high	mid	arr[mid]	Decision
1	0	6	3	27	27 < 34, so low = 4
2	4	6	5	48	48 > 34, so high = 4
3	4	4	4	34	Found at index 4

Example 1 – key present

```
Enter number of elements: 7
Enter 7 SORTED elements: 5 12 19 27 34 48 63
Enter element to search: 34

Element 34 found at index 4
```

Example 2 – key absent

```
Enter number of elements: 7
Enter 7 SORTED elements: 5 12 19 27 34 48 63
Enter element to search: 50

Element 50 not found.
```

Assignment 3: Bubble Sort and Count Swaps

Idea: In each pass, compare adjacent elements and swap them if they are in the wrong order; the largest remaining element “bubbles” to the end. A counter is incremented on every swap. A `swapped` flag stops early if the array is already sorted.

Program

```
#include <iostream>
using namespace std;

// Bubble Sort: repeatedly swap adjacent elements if out of order.
// Returns the total number of swaps performed.
int bubbleSort(int arr[], int n) {
    int swaps = 0;
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swaps++;
                swapped = true;
            }
        }
        if (!swapped) break;   // already sorted -> stop early
    }
    return swaps;
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[100];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    int swaps = bubbleSort(arr, n);

    cout << "\nSorted array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << "\nTotal swaps : " << swaps << endl;
    return 0;
}
```

Dry run: {64, 34, 25, 12, 22, 11}

Pass	Array after pass	Swaps in pass
1	34 25 12 22 11 64	5
2	25 12 22 11 34 64	4
3	12 22 11 25 34 64	3
4	12 11 22 25 34 64	1
5	11 12 22 25 34 64	1
	Total swaps	**14**

Example

```
Enter number of elements: 6
Enter 6 elements: 64 34 25 12 22 11

Sorted array: 11 12 22 25 34 64
Total swaps : 14
```

Assignment 4: Reverse an Array Without Extra Array

Idea: Use two pointers, `left` at the start and `right` at the end. Swap the two elements, move the pointers towards each other, and stop when they meet. Only one temporary variable is used, so extra space is O(1). For {10,20,30,40,50}: swap(10,50), swap(20,40), and 30 stays in the middle.

Program

```
#include <iostream>
using namespace std;

// Reverse in place using two pointers (no extra array).
void reverseArray(int arr[], int n) {
    int left = 0, right = n - 1;
    while (left < right) {
        int temp = arr[left];
        arr[left] = arr[right];
        arr[right] = temp;
        left++;
        right--;
    }
}

int main() {
    int n;
    cout << "Enter number of elements: ";
    cin >> n;
    int arr[100];
    cout << "Enter " << n << " elements: ";
    for (int i = 0; i < n; i++) cin >> arr[i];

    reverseArray(arr, n);

    cout << "\nReversed array: ";
    for (int i = 0; i < n; i++) cout << arr[i] << " ";
    cout << endl;
    return 0;
}
```

Example

```
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50

Reversed array: 50 40 30 20 10
```

Mini Project: Student Marks Analyzer

Task: Input marks of N students in an array, then sort and print the top 3 marks and the average score.
Approach: (1) read the marks and keep a running sum; (2) average = sum / N, using `double` to avoid integer division; (3) sort in descending order with bubble sort; (4) print the first three elements (or fewer if N < 3).

Program

```
#include <iostream>
using namespace std;

// Mini Project: Student Marks Analyzer
// 1. Input marks of N students  2. Sort (descending)
// 3. Print top 3 marks          4. Print average score
void sortDescending(int marks[], int n) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (marks[j] < marks[j + 1]) {
                int temp = marks[j];
                marks[j] = marks[j + 1];
                marks[j + 1] = temp;
            }
        }
    }
}

int main() {
    int n;
    cout << "Enter number of students: ";
    cin >> n;
    int marks[100];
    int sum = 0;

    for (int i = 0; i < n; i++) {
        cout << "Enter marks of student " << i + 1 << ": ";
        cin >> marks[i];
        sum += marks[i];
    }

    double average = (double)sum / n;
    sortDescending(marks, n);

    cout << "\n===== Student Marks Analyzer =====" << endl;
    cout << "Top marks:" << endl;
    int top = (n < 3) ? n : 3;
    for (int i = 0; i < top; i++)
        cout << "  " << i + 1 << ". " << marks[i] << endl;
    cout << "Average score: " << average << endl;
    return 0;
}
```

Example

```
Enter number of students: 6
Enter marks of student 1: 78
Enter marks of student 2: 92
Enter marks of student 3: 65
Enter marks of student 4: 88
Enter marks of student 5: 95
Enter marks of student 6: 71

===== Student Marks Analyzer =====
Top marks:
  1. 95
  2. 92
  3. 88
Average score: 81.5
```

Check: (78+92+65+88+95+71) / 6 = 489 / 6 = 81.5. Sorted: 95, 92, 88, 78, 71, 65, so the top 3 are 95, 92, 88.

Conclusion

All programs were compiled with g++ (C++17) and tested; the outputs shown above are actual program runs.